Airbnb Himeji: Zanzibar-Inspired Central Authorization

Relation Tuple Graph Modeling, Sub-10ms gRPC Checks & Multilayered Caching

SECURITY ARCHITECTURE

ACCESS CONTROL

1. The Problem Space: Authorization Sprawl & Microservice Deadlocks

In Airbnb's global marketplace, permissions are inherently transitive and relational:

- Can user U edit listing L? Yes, if U is the primary host, or an authorized co-host, or an administrator of a hospitality management company.
- Can user U view guest check-in codes for reservation R? Yes, if U is the primary booker, or an approved guest companion attached to the booking.

The Fall of Decentralized Microservice Authorization

Historically, authorization checks were scattered across hundreds of Ruby on Rails monolith models and downstream Java microservices. This triggered cascading RPC storms (10+ service calls per page load), inconsistent access enforcement, and severe tail latencies exceeding 200 milliseconds.

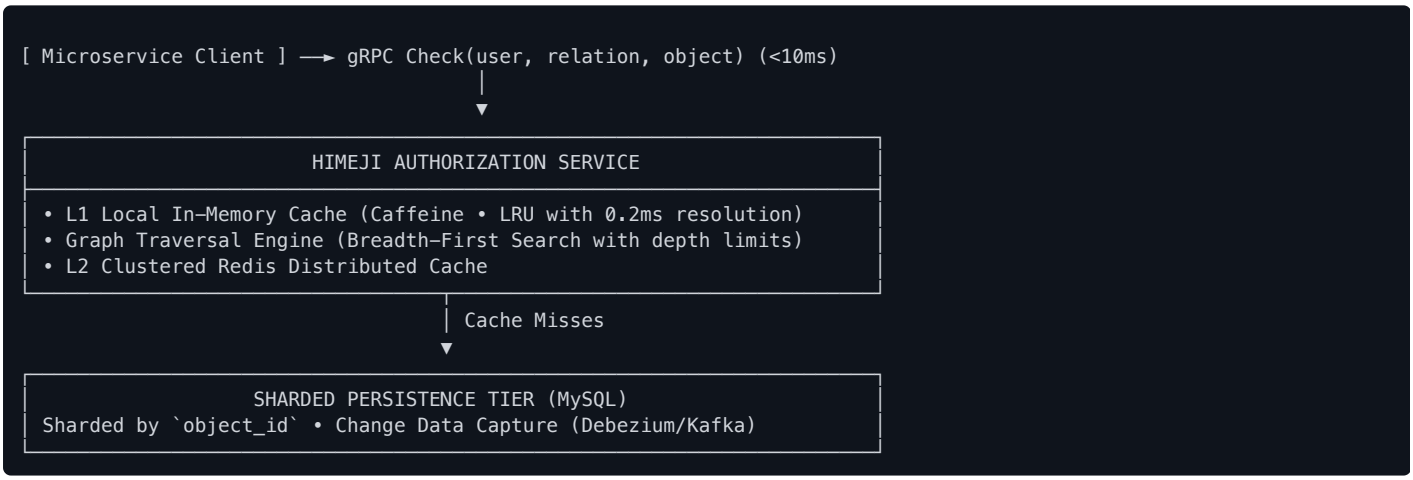
2. The Relation Tuple Graph Model (Google Zanzibar Inspired)

Airbnb replaced bespoke query logic with a unified relational graph represented as **Entity-Relation-Entity Tuples**:

```
            [ Object ] # [ Relation ] @ [ User or UserSet ]

Examples:
• listing:101#host@user:42           ── (User 42 is the direct host of listing 101)
• listing:101#co_host@user:88        ── (User 88 is an authorized co-host)
• reservation:500#parent@listing:101 ── (Reservation 500 belongs to listing 101)
• reservation:500#view@listing:101#host ── (Any host of listing 101 can view reservation)
```

3. Himeji System Topology



4. Graph Traversal Engine: Breadth-First Search & Safety Caps

When a service asks `Check(user:X, can_edit, listing:101)`, Himeji translates permission verification into directed graph reachability:

- **Direct Edge Evaluation:** Checks for exact match `listing:101#can_edit@user:X`. If present, returns `ALLOW` immediately.
- **Schema Rule Expansion:** Expands relational rules (e.g., `can_edit = host OR property_manager#admin`).
- **Breadth-First Search (BFS):** Traverses outward along edges. A visited-set bitmask and strict depth limit (e.g., max 6 hops) prevent infinite loops caused by circular administrative delegation.

5. Multilayered Caching & Event-Driven CDC Invalidation

L1 In-Memory + L2 Redis Caches
Local JVM/Go memory caches resolve over 98% of repeated authorization checks in under 0.2ms. Redis clusters cache computed permission sub-graphs.

Debezium & Kafka Invalidation
When host permissions change, MySQL writes to its binary log. Debezium streams CDC events to Kafka. Invalidation workers purge affected Redis keys within 50ms.

6. Architectural Comparison: Legacy RBAC vs. Himeji Zanzibar Model

Architectural Vector	Legacy Decentralized Auth	Airbnb Himeji Platform
Permission Representation	Hardcoded SQL joins across services	Unified Relation Tuples $\langle O \rangle \# \langle R \rangle @ \langle U \rangle$
Cross-Service Coupling	High (Cascading multi-hop RPCs)	Zero (Single gRPC call to Himeji)
Evaluation Latency	50ms – 250ms (Unpredictable)	**Sub-10ms p99 (Cache hit: <1ms)**
Transitive Relationships	Requires custom recursive application code	Native recursive graph traversal with depth caps

7. Fail-Closed vs. Fail-Open Operational Primitives

When network partitions or downstream database stalls occur, authorization systems cannot simply timeout. Himeji enforces strict semantic segregation based on action severity:

- **Mutating & Financial Actions (Fail-Closed):** Modifying bank payouts, canceling bookings, or altering property settings are strictly rejected (`DENY`) on timeout. Security and compliance take precedence over availability.
- **Public Read Actions (Graceful Degradation):** Displaying listing amenities or neighborhood reviews falls back to public visibility rather than failing the entire page render.

8. Key Architectural Takeaways for Enterprise Authorization

Core Principles Distilled from Airbnb Himeji

- **Unify Access Control as a Relational Graph:** Modeling permissions as relation tuples eliminates hundreds of fragmented SQL queries and establishes a single corporate source of truth.
- **Keep Microservices Out of Authorization Logic:** Applications should only ask binary questions: *"Can user X perform action Y on resource Z?"* The authorization platform encapsulates the complexity of resolving co-hosts and delegations.
- **Layer Caches with Event-Driven Invalidation:** In-process L1 memory caches combined with CDC-driven Kafka invalidations deliver sub-10ms evaluation while preserving strict consistency.
- **Guard Recursive Traversals with Strict Bounds:** Dynamic graph traversals must enforce visited bitmasks and depth limits to guarantee that circular permissions never trigger service crashes.